

Large Language Models: Tokens, Embeddings, Scaling and Decoding

Unit 3 of the Generative AI and LLM crash course · session 001 · curriculum node U3

GenAI crash course

2026-10-09

Contents

Session map	2
1. Terms: the term tree	3
2. Remember	4
3. Understand: the derivation	5
3.1 Tier 1 (easy): intuition and a picture	5
3.2 Tier 2 (medium): the definitions made precise	6
3.3 Tier 3 (medium-hard): the main derivations	6
3.3.1 Byte-pair encoding, step by step	6
3.3.2 Embeddings	7
3.3.3 Gradient of the loss with respect to the logits	7
3.4 Tier 4 (hard): decoding, the other half of the model	8
3.4.1 Temperature	8
3.4.2 Greedy, beam, top- k and top- p	9
3.4.3 How big, and how much data	10
3.5 Tier 5 (hardest): an open question	11
4. Apply	12
4.1 Worked problems	12
4.2 Code and figures	12
5. Analyze	12
5.1 Limits	13
5.2 Which term dominates?	13

5.3 Order-of-magnitude estimate	13
5.4 Failure modes	13
6. Evaluate	13
7. Create	14
8. Visualization lesson	14
9. Problem set	15
10. Retrieval practice and recap	15
11. Where next	15

Session map

Outcome of this session

After this session you can:

1. explain an LLM as one repeated operation, *predict a probability distribution over the next token*, and compute its training loss and perplexity by hand;
2. train a byte-pair-encoding (BPE) tokenizer on any text and say why subwords beat words and characters;
3. turn logits into a token with greedy, temperature, top- k , top- p and beam decoding, and predict what each knob does to the distribution;
4. estimate the parameter count, training FLOPs and compute-optimal data size of a model from its shape;
5. run, read and modify the code in `codes/llm/tokens_decoding/`, and reproduce all five figures.

- **Prerequisites:** Python and NumPy; probability (distributions, expectation, logarithms); a first neural-network course (a layer is a matrix times a vector, followed by a nonlinearity). Transformers are only *used* here as a black box that outputs logits; the box is opened in Unit 2.
- **Difficulty ladder used:** easy $\rightarrow$ medium $\rightarrow$ medium-hard $\rightarrow$ hard (Sections 3.1 to 3.4). The hardest tier is left as an open question in Section 3.5.
- **Deliverables:** this document (.tex, .pdf, .docx), 5 figures, the module `codes/llm/tokens_decoding/` (BPE, decoding, scaling law; 6 tests), and the interactive widgets *BPE stepper* and *Next-token sampler* in the LLM Studio playground.
- **Source anchors:** Sennrich, Haddow and Birch 2016, arXiv:1508.07909 (BPE for subword units); Holtzman et al. 2019, arXiv:1904.09751 (nucleus / top- p sampling); Hoffmann et al. 2022, arXiv:2203.15556 (compute-optimal training; the constants in Eq. (10) of that paper are the ones used in Section 5). All three IDs were checked against arXiv on the day of writing.

1. Terms: the term tree

Everything an LLM does rests on a short list of words. Each one is defined in Section 2 and used in Sections 3 and 4.

Term	Symbol	One-line meaning	Where it appears	New / known
token	x_i	the unit of text the model reads and writes (a word piece)	everywhere	new
vocabulary	V	the fixed set of tokens; its size is also written V	3.2	new
next-token prediction	$P(x_{n+1} x_{1:n})$	the single task an LLM is trained on	3.2	new
cross-entropy loss	$\mathcal{L}$	average of $-\ln$ (probability given to the true next token)	3.2	new
perplexity	PPL	$e^{\mathcal{L}}$: the effective number of equally likely choices	3.2	new
byte-pair encoding (BPE)		repeatedly merge the most frequent adjacent pair of symbols	3.3	new
embedding	E	a learned vector for every token id	3.3.2	new
logits	z	the model's raw scores, one per vocabulary entry	3.4.1	new
softmax	p_i	converts scores to a probability distribution	3.4.1	new
temperature	T	divides the logits before softmax; sharpens or flattens	3.4.1	new
greedy / beam decoding		pick the best token / keep the best B partial sequences	3.4.2	new
top- k / top- p	k, p	sample only from the k best / the smallest set with mass $\geq p$	3.4.2	new
scaling law	$L(N, D)$	loss as a function of parameters N and tokens D	3.4.3	new
context window	$n_{\max}$	the number of tokens the model can read at once	3.2	new

Terms *used but not taught here* (they are registered as later nodes): attention and the Transformer block (Unit 2), KV cache (Unit 10), instruction tuning (Unit 6).

2. Remember

Token and vocabulary

A **token** is a string chunk such as `low`, `est` or `the`. The **vocabulary** $\mathcal{V}$ is a fixed list of V tokens, each with an integer **id**. A text is turned into a list of ids by a **tokenizer**; the model only ever sees ids. Typical V for modern models is 3×10^4 to 2.5×10^5 .

Language model

A language model assigns a probability to every token sequence. By the chain rule of probability,

$$P(x_{1:n}) = \prod_{i=1}^n P(x_i \mid x_{1:i-1}), \quad (1)$$

so it is enough to learn one function: *given the prefix, output a distribution over the next token*. That function is **next-token prediction**.

Logits, softmax, temperature

The network outputs one real number per vocabulary entry, the **logits** $z \in \mathbb{R}^V$. The **softmax with temperature** $T > 0$ turns them into probabilities:

$$p_i(T) = \frac{e^{z_i/T}}{\sum_{j=1}^V e^{z_j/T}}. \quad (2)$$

$T = 1$ is the model's own distribution. $T < 1$ sharpens it, $T > 1$ flattens it.

Cross-entropy and perplexity

For a training text with true next tokens x_i , the **cross-entropy loss** is the mean negative log-probability the model gave to the truth, and the **perplexity** is its exponential:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n \ln p_{\theta}(x_i \mid x_{1:i-1}), \quad \text{PPL} = e^{\mathcal{L}}. \quad (3)$$

Memory hooks.

- *Token = syllable-ish*. Not letters, not words; the pieces that occur often enough to deserve their own id.
- *Logits are votes; softmax is the election; temperature is how sure the voters are*. Low T is a landslide, high T is a coin toss.
- *Perplexity = the model's effective branching factor*. $\text{PPL} = 10$ means “as unsure as a fair 10-sided die”.

Recall quiz (answers in Section 10).

1. Write the chain rule that lets a model with one output head define a probability for a whole sentence.

2. What does $T \rightarrow 0$ do to the sampled token? What does $T \rightarrow \infty$ do?
3. A model gives the true next token probability 0.25 at every step. What is its perplexity?
4. In BPE, which pair is merged at each step?
5. Does top- p keep a fixed number of tokens? Does top- k ?

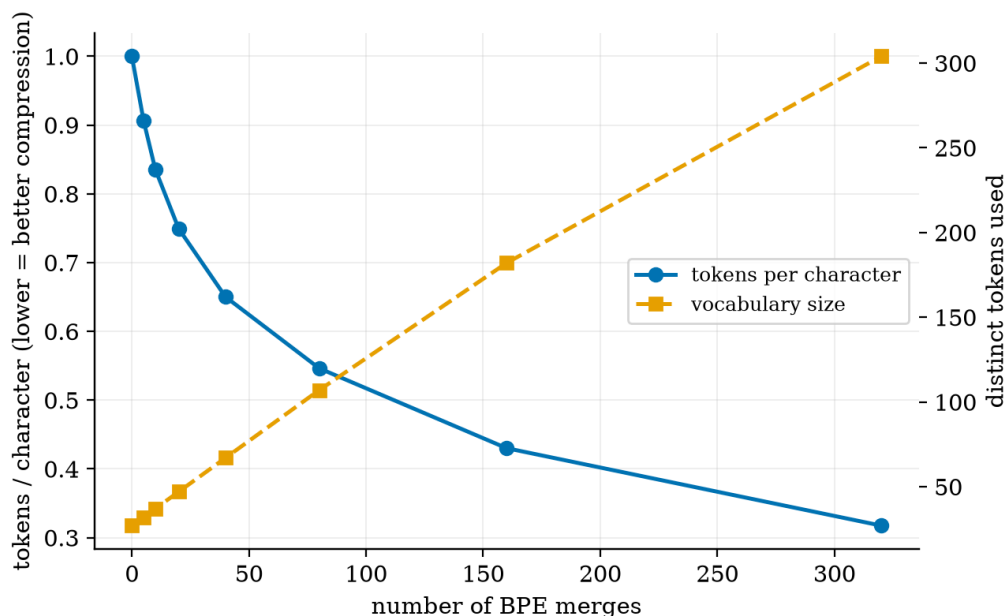
3. Understand: the derivation

3.1 Tier 1 (easy): intuition and a picture

Imagine a phone keyboard that suggests the next word. An LLM is that suggestion bar, scaled up until the suggestions are paragraphs. Two ideas do all the work.

1. **Text is cut into tokens, and each token becomes a vector.** The model never sees letters; it sees ids, and each id is looked up in a table of learned vectors (the embedding table).
2. **The model returns a probability for every possible next token, and a separate rule (decoding) chooses one.** The model is a *calculator of odds*; the decoding rule is the *decision*. The same model can sound careful or creative depending only on that rule.

Writing a long answer is then a loop: predict, choose, append the chosen token to the prefix, predict again. This is why an LLM answer appears one token at a time, and why a bad early choice cannot be taken back.



Why subwords: as BPE learns more merges, the same text needs fewer tokens (blue, left axis) while the vocabulary grows (orange, right axis). Zero merges is character-level: 1 token per character.

Figure 1 is the whole motivation for tokenization. Character-level text is short in vocabulary (27 symbols here) but long in sequence length; whole-word text is short in sequence length but has an unbounded vocabulary and cannot spell a word it has never seen. BPE sits in between and lets you choose where.

3.2 Tier 2 (medium): the definitions made precise

The model. Write the whole network as a function f_θ that maps a prefix $x_{1:i-1}$ (at most $n_{\max}$ tokens, the **context window**) to logits $z \in \mathbb{R}^V$. Then

$$p_\theta(x_i = v \mid x_{1:i-1}) = \text{softmax}(f_\theta(x_{1:i-1}))_v.$$

Training. Take a large text, cut it into sequences of tokens, and for each position make the model predict the true next token. Minimise (3) with gradient descent. There are no human labels: the text *is* the label (this is called self-supervised learning). Because the sum runs over every position of every sequence, one sequence of length n supplies n training examples.

Worked example: loss and perplexity. Suppose on a four-token sentence the model gave the true tokens probabilities 0.5, 0.25, 0.125, 0.5. Then

$$\begin{aligned}\mathcal{L} &= \frac{1}{4}(\ln 2 + \ln 4 + \ln 8 + \ln 2) = \frac{1}{4} \ln 128 = 1.2128, \\ \text{PPL} &= e^{\mathcal{L}} = 128^{1/4} = 3.364.\end{aligned}$$

The perplexity is the geometric mean of $1/p$: the model was, on average, as unsure as a fair die with about 3.4 faces. A model that guesses uniformly over a vocabulary of $V = 50,000$ has $\text{PPL} = 50,000$; every real improvement is a reduction from that number.

What the loss number means

Loss $\mathcal{L}$ is in **nats** (natural log). Dividing by $\ln 2$ gives **bits per token**. Perplexity $= e^{\mathcal{L}}$. A drop of 0.1 nats is a 10% cut in perplexity, and it is a large improvement in practice.

3.3 Tier 3 (medium-hard): the main derivations

3.3.1 Byte-pair encoding, step by step

BPE (Sennrich et al. 2016) starts from characters and learns a merge table from data.

Algorithm.

1. Split every word into characters and append an end-of-word marker `</w>`. Count how often each word occurs.
2. Count every adjacent pair of symbols, weighted by word frequency.
3. Merge the most frequent pair into a single new symbol everywhere. Record the merge.
4. Repeat steps 2-3 for M merges. M is the one hyperparameter; final vocabulary $\approx$ (initial characters) $+ M$.

To encode new text, split into characters and replay the recorded merges **in the order they were learned**.

Worked example (this is exactly what `train_bpe` prints for the toy corpus `low×5, lower×2, newest×6, widest×3`):

first six merges. Step 1 counts $\mathbf{t} < / \mathbf{w} >$ as 6 (newest) + 3 (widest) = 9. The suffix **est** emerges *by itself* from frequency, with no linguistic rule.

step	merged pair	count	new symbol
1	$\mathbf{t} + < / \mathbf{w} >$	9	$\mathbf{t} < / \mathbf{w} >$
2	$\mathbf{s} + \mathbf{t} < / \mathbf{w} >$	9	$\mathbf{st} < / \mathbf{w} >$
3	$\mathbf{e} + \mathbf{st} < / \mathbf{w} >$	9	$\mathbf{est} < / \mathbf{w} >$
4	$\mathbf{o} + \mathbf{w}$	7	$\mathbf{ow}$
5	$\mathbf{l} + \mathbf{ow}$	7	$\mathbf{low}$
6	$\mathbf{w} + \mathbf{est} < / \mathbf{w} >$	6	$\mathbf{west} < / \mathbf{w} >$

Encoding the unseen word **lowest** with these merges gives $\mathbf{low} + \mathbf{est} < / \mathbf{w} >$: a word that never occurred in training is spelled from pieces that did. That is why modern LLMs never meet an “unknown word”. (Byte-level BPE, used by GPT-style models, starts from the 256 byte values instead of characters, so *any* string is representable.)

Common mistake

BPE merges are **ordered**. Applying them in a different order, or greedily by longest match, gives different tokens from the training tokenizer. The same text then lands on token ids the model never saw in that form.

3.3.2 Embeddings

A token id v selects row v of a learned matrix $E \in \mathbb{R}^{V \times d}$: $e_v = E_{v,:}$. Equivalently, multiply the one-hot vector $\mathbf{1}_v \in \mathbb{R}^V$ by E :

$$e_v = \mathbf{1}_v^\top E.$$

The d numbers in e_v are free parameters learned by gradient descent, with no meaning at the start. After training, tokens used in similar contexts have vectors pointing in similar directions, and the usual measure of “similar direction” is the **cosine similarity**

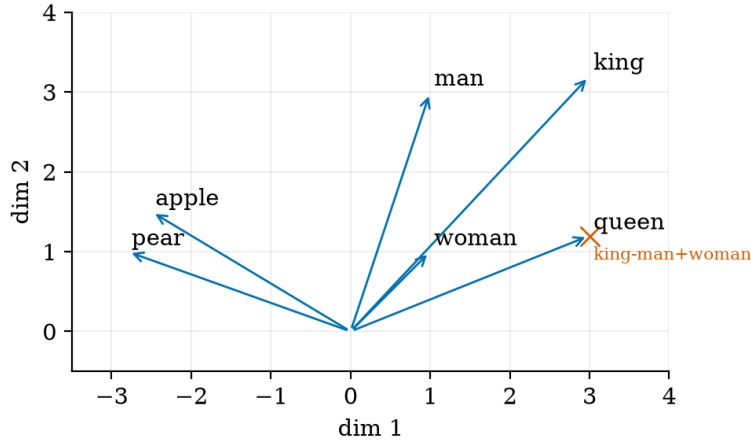
$$\cos(u, v) = \frac{u^\top v}{\|u\| \|v\|} \in [-1, 1]. \quad (4)$$

The figure is a *constructed* example: I placed the vectors so that the king-queen offset equals the man-woman offset, so the analogy is exact by design (error 2×10^{-16}). In real models the analogy holds approximately and only for some relations; do not treat it as a law.

3.3.3 Gradient of the loss with respect to the logits

This one result explains why training is stable and simple. For one position with true token y and logits z (take $T = 1$), the loss is $\ell = -\ln p_y$ with $p_i = e^{z_i} / S$, $S = \sum_j e^{z_j}$.

$$\begin{aligned}
 \ell &= -z_y + \ln S && (\text{since } \ln p_y = z_y - \ln S) \\
 \frac{\partial \ell}{\partial z_i} &= -\delta_{iy} + \frac{1}{S} \frac{\partial S}{\partial z_i} && (\text{derivative of } \ln S) \\
 &= -\delta_{iy} + \frac{e^{z_i}}{S} && (\text{only the term } j = i \text{ depends on } z_i) \\
 &= p_i - \delta_{iy}.
 \end{aligned} \quad (5)$$



Toy two-dimensional embeddings (built by hand to show the idea, not learned). king and queen point the same way (cosine 0.906), apple and pear almost the same (0.981), king and apple are unrelated (-0.211). The red cross is king - man + woman, which lands exactly on queen.

In vector form the gradient is simply $p - \mathbf{1}_y$: **predicted distribution minus the one-hot truth**. It pushes the true logit up by $1 - p_y$ and every other logit down by p_i . It is bounded by 1 in each component, so a confidently wrong prediction cannot produce an exploding gradient at the output.

3.4 Tier 4 (hard): decoding, the other half of the model

The network has produced logits z . *Decoding* is the rule that turns them into the next token. Each strategy below is a different answer to one tension: **quality** (pick likely tokens) against **diversity** (do not repeat yourself).

3.4.1 Temperature

Let $p_i(T)$ be as in (2) and let $i^* = \arg \max_i z_i$ be the unique largest logit. Divide numerator and denominator by $e^{z_{i^*}/T}$:

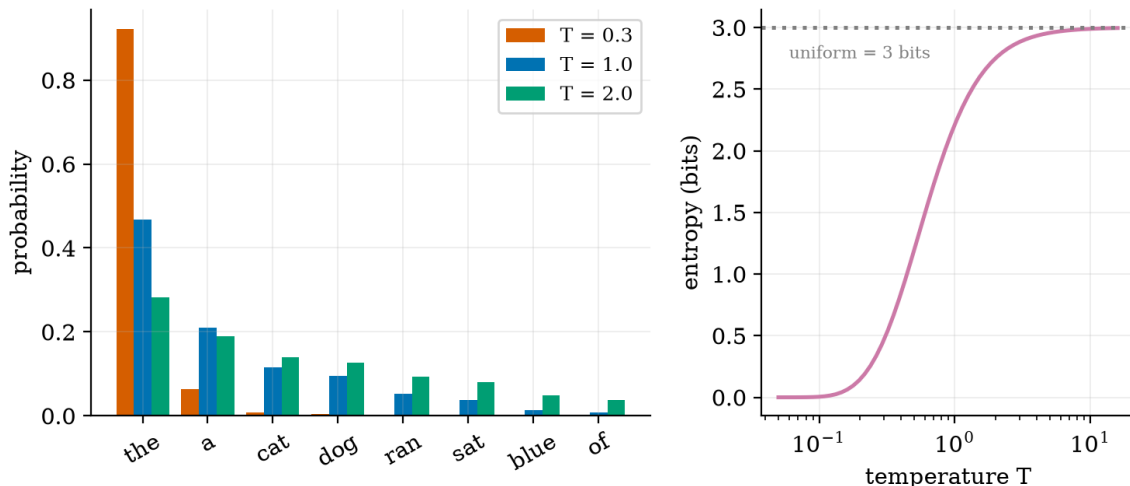
$$p_i(T) = \frac{e^{(z_i - z_{i^*})/T}}{1 + \sum_{j \neq i^*} e^{(z_j - z_{i^*})/T}}.$$

Every exponent $(z_j - z_{i^*})/T$ with $j \neq i^*$ is negative. As $T \rightarrow 0^+$ it tends to $-\infty$, so $p_{i^*} \rightarrow 1$ and all others $\rightarrow 0$: **greedy**. As $T \rightarrow \infty$ every exponent tends to 0, so $p_i \rightarrow 1/V$: **uniform**. In between, T rescales the *gaps* between logits: doubling T halves every gap.

For these eight logits the numbers (from `fig02_temperature.py`) are:

lowering T from 1 to 0.3 nearly doubles the chance of the top token and cuts the entropy by a factor of almost five.

T	$p(\text{the})$	entropy (bits)
0.3	0.922	0.465
1.0	0.467	2.214
2.0	0.282	2.751



Left: the same eight logits at three temperatures. Right: entropy (in bits) against T ; it rises from 0 toward the uniform value of 3 bits.

3.4.2 Greedy, beam, top- k and top- p

Greedy takes $\arg \max_i p_i$ every step. It is fast and deterministic, but it is *myopic*: the best token now can lead to a worse sentence. **Beam search** keeps the B highest-scoring partial sequences, where a sequence's score is the sum of its token log-probabilities $\sum_i \ln p_i$, and extends all of them each step.

Worked example: greedy loses. Token 0 is the start; the next-token table is

$$P = \begin{pmatrix} 0 & 0.6 & 0.4 \\ 0.5 & 0.25 & 0.25 \\ 0.01 & 0.01 & 0.98 \end{pmatrix}, \quad \text{row} = \text{previous token.}$$

Greedy for two steps picks token 1 (0.6) and then token 0 (0.5): probability $0.6 \times 0.5 = 0.300$ ($\ln = -1.204$). The path $0 \rightarrow 2 \rightarrow 2$ has probability $0.4 \times 0.98 = 0.392$ ($\ln = -0.936$). Beam search with $B = 3$ finds it; greedy cannot, because it commits to the 0.6 branch. The test `test_beam_beats_greedy_on_trap` checks this.

Beam search is not the best sampler for open-ended text

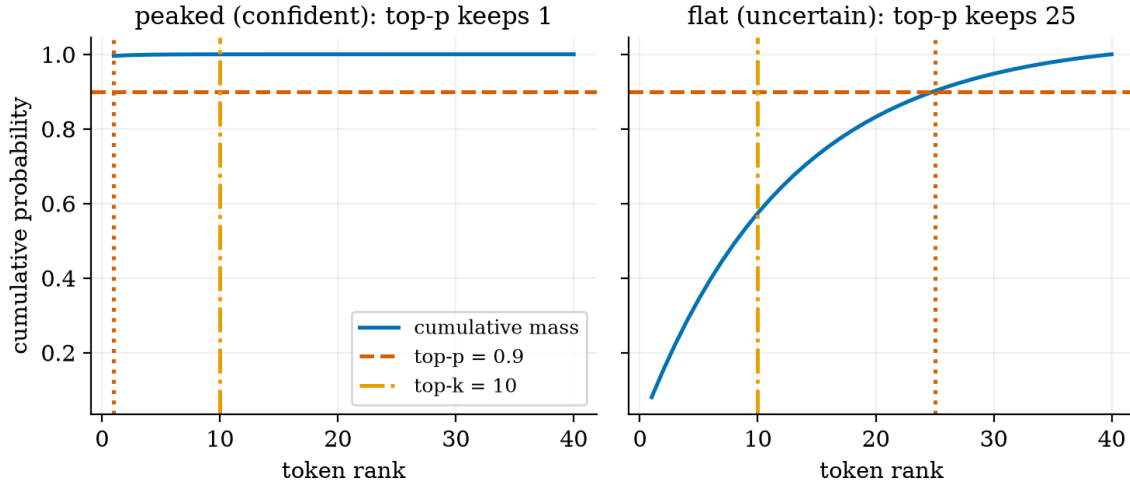
Holtzman et al. (2019) showed that maximising likelihood on open-ended generation produces bland, repetitive text, which is why chat models usually *sample* rather than use beam search. Beam search stays the standard for tasks with one right answer, such as translation or code.

Top- k . Keep the k most probable tokens, set the rest to zero, renormalise, then sample.

Top- p (nucleus). Sort tokens by probability and keep the smallest prefix whose cumulative mass is at least p :

$$\mathcal{V}_p = \text{the smallest set } S \text{ with } \sum_{i \in S} p_i \geq p, \quad q_i = \frac{p_i \mathbf{1}[i \in \mathcal{V}_p]}{\sum_{j \in \mathcal{V}_p} p_j}. \quad (6)$$

Worked example: $p = (0.5, 0.3, 0.15, 0.05)$ and $p_{\text{top}} = 0.8$. The first two tokens carry $0.5 + 0.3 = 0.8 \geq 0.8$, so the nucleus has size 2 and $q = (0.625, 0.375, 0, 0)$. For top- $k = 2$ the same vector is kept, but the *rule* is different:



Cumulative probability against token rank. When the model is confident (left) top- $p = 0.9$ keeps 1 token; when it is unsure (right) it keeps 25. Top- $k = 10$ keeps 10 in both cases.

Why top- p is adaptive

Top- k keeps a **fixed number** of tokens, so it can be too loose when the model is sure and too tight when it is not. Top- p keeps a **fixed amount of probability**, so the number of tokens adapts to the model's confidence (1 versus 25 in Figure 3).

The standard order. Temperature is applied to logits first, then the top- k and top- p filters, then sampling. Changing the order changes the answer, because the filters work on probabilities that already depend on T .

3.4.3 How big, and how much data

Counting parameters. A decoder-only Transformer with L blocks of width d has, per block, $4d^2$ attention weights (query, key, value and output matrices, each $d \times d$) and $8d^2$ in the feed-forward layer ($d \rightarrow 4d \rightarrow d$). Ignoring small terms (biases, layer norms), plus the token embedding table,

$$N \approx 12 L d^2 + V d. \quad (7)$$

For the smallest GPT-2 ($L = 12$, $d = 768$, $V = 50,257$): $12 \cdot 12 \cdot 768^2 = 84,934,656$ and $50,257 \cdot 768 = 38,597,376$, a total of $123,532,032 \approx 1.24 \times 10^8$, consistent with the commonly quoted 124M. (I derived this from the formula; I did not download the model to count its tensors.)

Counting training compute. One forward pass costs about one multiply and one add per weight per token, so $2N$ FLOPs per token. The backward pass costs about twice the forward pass. Therefore

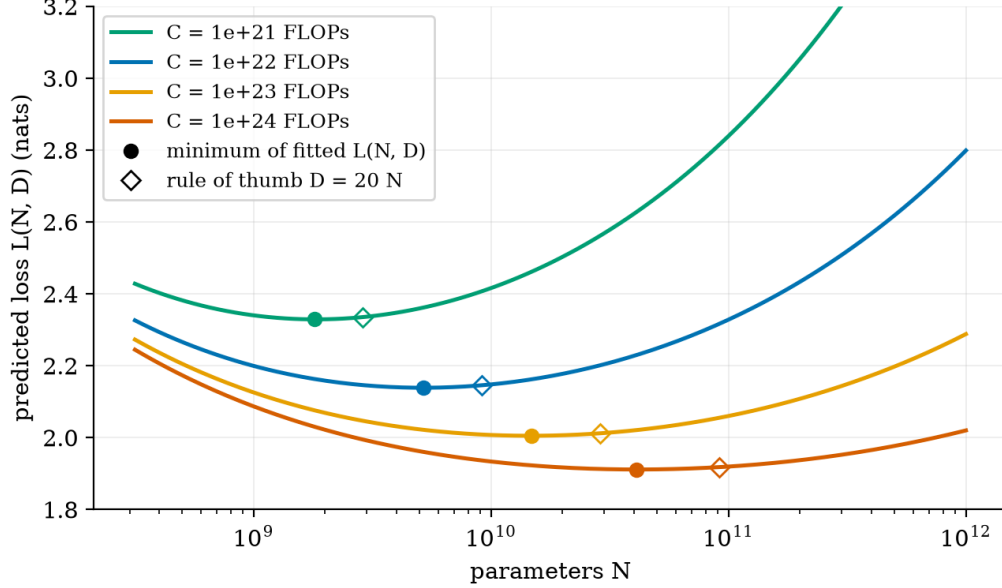
$$C \approx (2N + 4N) D = 6 N D \quad \text{FLOPs}, \quad (8)$$

where D is the number of training tokens. This approximation ignores the attention score computation, which matters only for very long contexts.

The scaling law. Hoffmann et al. (2022) fit the final loss as

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \quad E = 1.69, A = 406.4, B = 410.7, \alpha = 0.34, \beta = 0.28 \quad (9)$$

(their Eq. (10), values checked against the paper). Read it term by term: E is the *irreducible* loss (the entropy of text itself), A/N^α is the penalty for a model that is too small, and B/D^β is the penalty for too little data. For a fixed budget $C = 6ND$, setting $D = C/(6N)$ turns L into a function of N alone, with a minimum where the two penalties balance.



IsoFLOP curves from Eq. (10) with $D = C/(6N)$. Each curve has one best model size (filled dot). The diamonds mark the rule of thumb $D = 20 N$; the two differ by under 0.01 nats because the minimum is flat.

3.5 Tier 5 (hardest): an open question

Figure 4 contains a result that surprised me while writing it, so I report it as it is. Minimising Eq. (10) at each budget gives these data-to-parameter ratios (from `fig04_scaling.py`):

the minimum of the fitted formula does **not** give $D/N = 20$, and the ratio grows with C .

budget C (FLOPs)	best N	best D	D/N
10^{21}	1.8×10^9	9.2×10^{10}	51
10^{22}	5.2×10^9	3.2×10^{11}	62
10^{23}	1.5×10^{10}	1.1×10^{12}	76
10^{24}	4.1×10^{10}	4.1×10^{12}	100

This is not a bug. The paper itself reports that this parametric fit (its “approach 3”) gives $N_{\text{opt}} \propto C^{0.46}$ and $D_{\text{opt}} \propto C^{0.54}$, so $D/N \propto C^{0.08}$ rises slowly, while its other two approaches give exponents near 0.50, i.e. a constant ratio, which is where the famous “about 20 tokens per parameter” comes from. Which exponent is right, and how much of the disagreement is noise in the fit, is an open empirical question; I have not resolved it here. **For planning, the rule of thumb and the fit agree on loss to within 0.01 nats (Figure 4), so the choice matters little for the loss but a lot for how large a model you must serve.**

4. Apply

4.1 Worked problems

Problem 1 (tokens and cost). A tokenizer averages about 0.3 tokens per character on technical English (Figure 1 shows 0.317 after 320 merges on this course’s syllabus). How many tokens is a 6{,}942-character page? Answer: $6,942 \times 0.317 \approx 2,200$ tokens (the run gives 2,204). A 4{,}096-token context window therefore holds about two such pages.

Problem 2 (size and compute of GPT-2 small). From (7), $N \approx 1.24 \times 10^8$. Using $D = 20N = 2.5 \times 10^9$ tokens, $C = 6ND \approx 1.8 \times 10^{18}$ FLOPs. A GPU sustaining 10^{14} FLOP/s would need about 1.8×10^4 s, i.e. 5 hours, at perfect efficiency; real utilisation is far lower.

Problem 3 (temperature by hand). Logits $z = (2, 1, 0)$. At $T = 1$, $p \propto (e^2, e^1, e^0) = (7.389, 2.718, 1)$ with sum 11.107, so $p = (0.665, 0.245, 0.090)$. At $T = 0.5$, $p \propto (e^4, e^2, 1) = (54.6, 7.389, 1)$, sum 62.99, so $p = (0.867, 0.117, 0.016)$: the top token rose from 0.665 to 0.867.

4.2 Code and figures

Everything is in `codes/llm/tokens_decoding/` (see its README). The core of decoding is four lines:

```
def softmax(z, T=1.0):
    z = np.asarray(z, dtype=float) / T
    z = z - z.max()                # subtract max: same answer, no overflow
    e = np.exp(z)
    return e / e.sum()

def sample(logits, T=1.0, top_k=None, top_p=None, rng=None):
    p = softmax(logits, T)        # 1. temperature
    if top_k is not None: p = top_k_filter(p, top_k)  # 2. keep k best
    if top_p is not None: p = top_p_filter(p, top_p)  # 3. keep the nucleus
    return int(rng.choice(len(p), p=p))  # 4. draw one token
```

The BPE trainer is `train_bpe(text, num_merges)`; it returns the ordered merge list and a history of (pair, count, vocabulary size). To reproduce Figure 1 run `figures/fig01_bpe.py`, which trains on the Syllabus text with 0 to 320 merges. All five figure scripts print the numbers quoted in this document.

Try it in the playground

Open the **BPE stepper** and press *Merge* to watch the table above appear one row at a time. Open the **Next-token sampler** and drag temperature, top- k and top- p while watching which bars survive. Both are linked from the Unit 3 page of the site.

5. Analyze

Checklist items used: limits (5.1), dominance (5.2), order-of-magnitude estimate (5.3), failure modes (5.4).

5.1 Limits

- $T \rightarrow 0$: greedy (Section 3.4.1). $T \rightarrow \infty$: uniform over all V tokens, i.e. garbage. Useful range is roughly $0.2 \leq T \leq 1.5$.
- $k = 1$ or $p \rightarrow 0$: greedy. $k = V$ or $p = 1$: no filtering, only temperature remains.
- Merges $M \rightarrow 0$: character-level. $M \rightarrow \infty$: every training word is one token. Figure 1 shows the sequence length falling and the vocabulary rising smoothly between the two.

5.2 Which term dominates?

In (7), $12Ld^2$ exceeds the embedding term Vd when $12Ld > V$. For GPT-2 small, $12 \cdot 12 \cdot 768 = 110,592 > 50,257$ so the blocks dominate (69%), but the embedding table is still 31% of the parameters. For a tiny model it can be most of them.

5.3 Order-of-magnitude estimate

At $N = 7 \times 10^{10}$ and $D = 1.4 \times 10^{12}$ (the size the Chinchilla paper trains), $C = 6ND \approx 5.9 \times 10^{23}$ FLOPs. The code `compute_optimal(5.76e23)` returns $N = 6.9 \times 10^{10}$ and $D = 1.39 \times 10^{12}$ under the $D = 20N$ rule, matching that scale (the test `test_compute_optimal_matches_chinchilla_scale` checks it).

5.4 Failure modes

- **Spelling and arithmetic.** A model sees `strawberry` as a few subword tokens, not letters, so letter counting is genuinely hard for it. This is a tokenizer property, not stupidity.
- **Whitespace and language bias.** A tokenizer trained mostly on English splits other languages into many more tokens, which makes them slower and more expensive for the same content.
- **Repetition loops under greedy or low- T decoding.** Once a phrase is likely, repeating it is the most likely continuation, and nothing breaks the cycle.
- **Numerical overflow.** Exponentiating large logits overflows; subtracting the maximum (as in the code) leaves the result unchanged and avoids it.

6. Evaluate

Tokenization choices. Characters: tiny vocabulary, long sequences, slow; words: short sequences, huge vocabulary, out-of-vocabulary failures; BPE/SentencePiece/byte-level: the pragmatic middle, and now universal. The cost is the oddities in 5.4.

Decoding choices.

rules of thumb, not laws. Always test on your own task.

Task	Good choice	Why
translation, code, extraction	greedy or beam, low T	one right answer, want consistency
chat, brainstorming	$T \approx 0.7$ -1.0, top- $p \approx 0.9$	variety without nonsense
creative writing	higher T , top- p	explore the tail

Task	Good choice	Why
------	-------------	-----

When *not* to trust these numbers. The scaling constants are fitted to one family of models and one data distribution; they are not constants of nature. Do not use Eq. (10) to predict a model with a different architecture, data mix or training recipe. A lower loss does not guarantee better usefulness: Unit 9 treats evaluation.

Wider context. A language model is a very large compression machine: the loss in bits per token is the code length needed to store the text, so “predicting the next word well” and “compressing text well” are the same problem (Shannon, 1948). Whether prediction alone gives understanding is a live philosophical debate that this course does not settle.

7. Create

What you can build from this

1. **Train your own tokenizer** on a folder of your own notes with `train_bpe`; plot tokens per character against merges for *your* text and compare with Figure 1.
2. **A temperature explorer** that prints ten samples at five temperatures from a small bigram model built from any text file.
3. **A beam-versus-greedy trap finder:** generate random 3×3 tables and count how often beam search ($B = 3$) beats greedy.
4. **Combine with Unit 2:** replace the bigram table with the self-attention model you build there, and keep the same `sample` function.
5. **Module for the open-source package:** extend `tokens_decoding` with repetition penalty and min- p filtering, with tests.

8. Visualization lesson

charts taught here. How to read each: in the cumulative curve, find where it crosses your threshold and read the rank on the x-axis; in the IsoFLOP plot, the minimum of each curve is the best model size for that budget.

Chart type	Best for	How it misleads	matplotlib call
dual-axis line	two quantities that share an x (Fig. 1)	the two scales can be stretched to suggest a relationship	<code>ax.twinx()</code>
grouped bar	the same categories under three settings (Fig. 2 left)	colours too similar, or a truncated y-axis	<code>ax.bar(x+offset, h, width)</code>
semilog line	quantities spanning orders of magnitude (Fig. 2 right, Fig. 4)	straight lines suggest power laws that may not exist	<code>ax.semilogx(x, y)</code>
cumulative curve	“how many items cover p of the mass” (Fig. 3)	a steep start hides the long tail	<code>ax.plot(rank, np.cumsum(p))</code>

Chart type	Best for	How it misleads	matplotlib call
vector (arrow) plot	embeddings and directions (Fig. 5)	two dimensions cannot show the geometry of $4\{, \}096$	<code>ax.annotate("", xy, xytext, arrowprops=...)</code>

9. Problem set

Graded exercises

1. (easy) Compute perplexity for a model that assigns probabilities 0.1, 0.1, 0.1 to the true tokens of a three-token text.
2. (medium) With logits $(3, 1, 0, -1)$, compute p at $T = 1$ and at $T = 2$, and the top- $p = 0.9$ nucleus at $T = 1$.
3. (medium-hard) Run `train_bpe` on a paragraph of your choice for 30 merges. List the first five merges and explain each in terms of frequency.
4. (hard) Prove that for any logits the entropy of $\text{softmax}(z/T)$ is non-decreasing in T . (Hint: differentiate the entropy with respect to $1/T$.)
5. (hardest) Use Eq. (10) to derive $N_{\text{opt}}(C)$ and show analytically that $N_{\text{opt}} \propto C^{\beta/(\alpha+\beta)}$. Compare with the exponent 0.46 reported in the paper, and explain why D/N is not constant.

10. Retrieval practice and recap

Answers to the Section 2 quiz.

1. $P(x_{1:n}) = \prod_i P(x_i | x_{1:i-1})$, Eq. (1).
2. $T \rightarrow 0$: the top token gets all the probability (greedy). $T \rightarrow \infty$: uniform.
3. $\mathcal{L} = \ln 4$, so PPL = 4.
4. The most frequent adjacent pair, counted over the whole corpus, weighted by word frequency.
5. Top- k keeps exactly k ; top- p keeps a variable number set by cumulative mass.

Explain it to a friend.

- Why can a chatbot give two different answers to the same question?
- Why does an LLM sometimes miscount the letters in a word?
- What is the difference between “the model” and “the decoding strategy”?

Re-derive tomorrow without looking: (a) the gradient $p - \mathbf{1}_y$ ((5)); (b) the $T \rightarrow 0$ and $T \rightarrow \infty$ limits of the softmax; (c) $N \approx 12Ld^2 + Vd$ and $C \approx 6ND$.

11. Where next

- **Next node:** Unit 2 (Transformer architecture), which opens the black box f_θ used throughout this session; then Unit 4 (prompting) uses the decoding knobs from Section 3.4.2.

- **Terms met but not taught:** attention, KV cache, instruction tuning, SentencePiece, repetition penalty.
- **Open threads:** the D/N exponent question in Section 3.5; whether prediction implies understanding (Section 6).